PROGREE

Data Science Internship

Tasks 2, 3 & 4

Automated Web Scraping · Customer Segmentation · Time-Series Forecasting

Submitted by: Aleesha Bukhari

Github: aleesha-10

GitHub Repository: https://github.com/aleesha-10/internships/tree/main/Progree

2025

At a Glance

2	100	4	5.35%	lag_12
Sites Scraped	Quotes Extracted	Customer Segments	Forecast MAPE	Top SHAP Feature

Overview

This report documents three end-to-end data science projects completed as part of the Progree Data Science Internship. Each task was implemented as a self-contained, fully-executed Jupyter notebook available in the GitHub repository. The work spans three core disciplines: data engineering (automated web scraping), unsupervised machine learning (customer segmentation), and time-series forecasting with model interpretability. All pipelines go beyond the minimum brief requirements — they include algorithm comparisons, interactive visualizations, baseline benchmarking, and business-oriented output framing.

Task 2 — Automated Web Scraping & Data Extraction Pipeline

A fault-tolerant crawler validated against two independent live sites

Primary Source	quotes.toscrape.com — 10 pages, 100 records
Second Source	books.toscrape.com — structural generalization proof
Stack	requests · BeautifulSoup · pandas · sqlite3 · plotly
Resilience	Retry logic with exponential backoff + jitter on every request
Outputs	quotes_dataset.csv · quotes_dataset.db · books_dataset.csv

Methodology

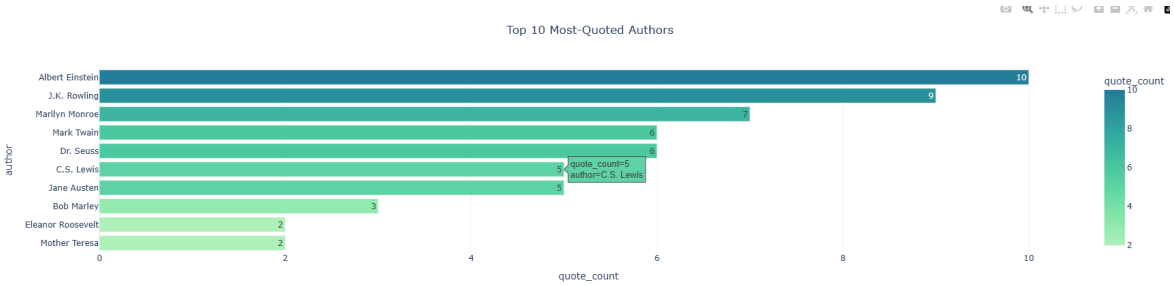
The pipeline follows a clean separation of concerns across four stages: fetch, parse, clean, and persist. Every HTTP request is wrapped in a retry + exponential backoff function to handle dropped connections and rate-limit responses gracefully — the kind of production resilience that separates a throwaway script from a real data pipeline. Automated pagination follows the 'Next' link on each page until it no longer exists, requiring zero manual configuration of page counts.

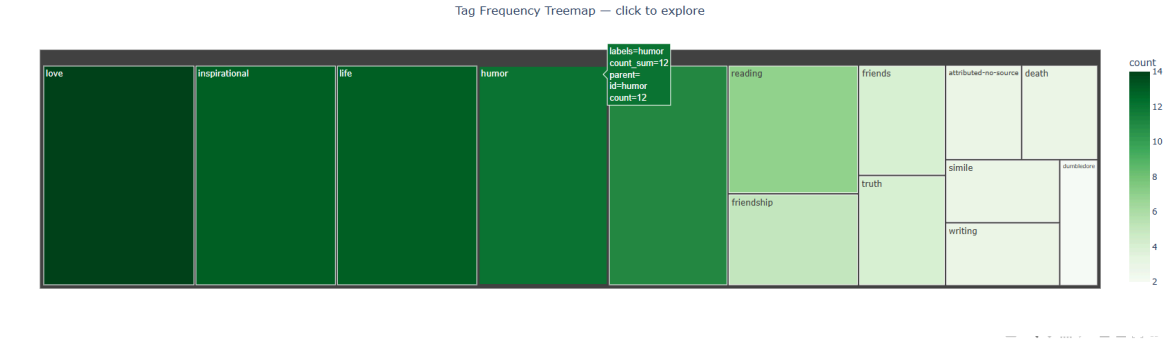
```
fetch_with_retries(url, max_retries=3, base_delay=1.0)
  → For each attempt: GET url, on failure wait base_delay * 2^(attempt-1) + jitter
fetch_all_pages(base_url)  → follows 'Next' links automatically until exhausted
parse_page(html)           → extracts quote_text, author, author_url, tags per card
clean_text(value)          → strips smart-quotes and normalises whitespace
df  →  quotes_dataset.csv  +  quotes_dataset.db (table: quotes)
```

Results

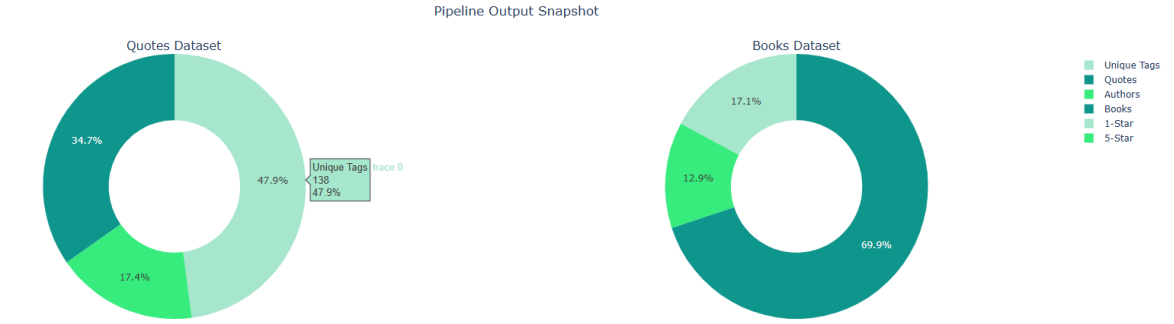
10	100	0	2
Pages Crawled	Records Extracted	Duplicates Removed	Sources Validated

- **Top authors by quote count:** Albert Einstein (10), J.K. Rowling (9), Marilyn Monroe (7), Dr. Seuss (6), Mark Twain (6) — confirmed identical across CSV and SQLite reloads.
- The pipeline was pointed at a structurally unrelated second site (books.toscrape.com) with a new parse_books_page() function but identical crawl → clean → persist architecture, proving the pattern generalizes beyond hardcoded selectors.
- Interactive Plotly analytics layer built inline: top-author bar chart, tag-frequency treemap, price-by-rating box plot, and a pipeline output snapshot.
- A structured JSON run log is printed at the end of every run — timestamp, pages crawled, records extracted, duplicates removed, and output files.
- Both CSV and SQLite outputs pass a programmatic row-count assertion before the notebook completes.


Top 10 Most-Quoted Authors
Albert Einstein
J.K. Rowling
Marilyn Monroe
Mark Twain
Dr. Seuss
C.S. Lewis
Jane Austen
Bob Marley
Eleanor Roosevelt
Mother Teresa
author
quote_count
quote_count=5
author=C.S. Lewis


Tag Frequency Treemap — click to explore
love
inspirational
life
humor
reading
friends
attributed-no-source
death
simile
dumbledore
truth
friendship
writing
labels=humor
count_sum=12
parent=
id=humor
count=12
count


Book Price Distribution by Star Rating
Price (£)
Star Rating
(3, max: 57.31)
(3, upper fence: 57.31)
(3, q3: 48.19)
(3, median: 35.07)
(3, q1: 27.865)
(3, lower fence: 10.16)
(3, min: 10.16)


Pipeline Output Snapshot
Quotes Dataset
34.7%
47.9%
17.4%
Unique Tags
138
47.9%
trace 0
Books Dataset
17.1%
12.9%
69.9%
Unique Tags
Quotes
Authors
Books
1-Star
5-Star

Task 3 — Customer Segmentation (Clustering)

K-Means + Hierarchical comparison on real Mall Customer data — with 3D visuals and business personas

Dataset	Mall Customer Segmentation (Kaggle) — 200 real customers
Features	Age · Annual Income (k$) · Spending Score (1–100)
Methods	StandardScaler → PCA → Elbow/Silhouette → K-Means + Hierarchical (Ward)
Optimal k	4 clusters (confirmed by both Elbow Method and silhouette score peak)
Stack	scikit-learn · scipy · plotly · seaborn
Output	customer_segments.csv — 200 labeled customer records

Methodology

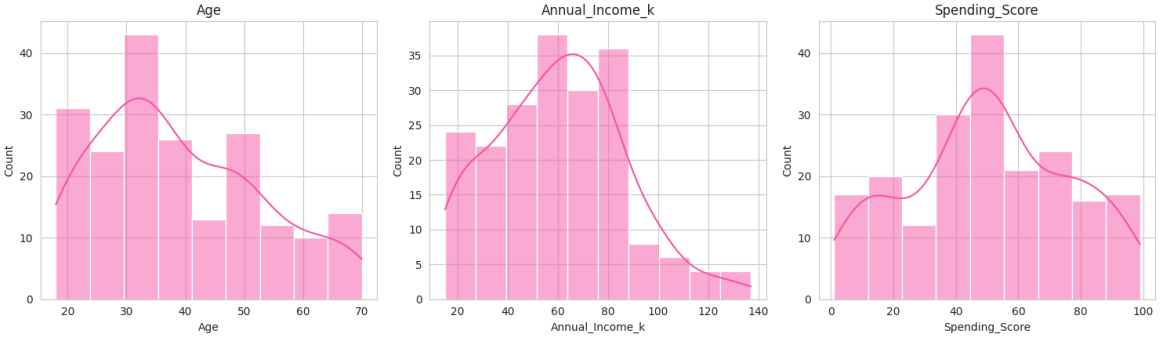
Features were standardized with StandardScaler before any distance-based computation. PCA reduced the three features to two principal components for visualization. The Elbow Method and silhouette score were both computed across k = 2–10; both metrics converged independently on k = 4. A final K-Means model was then cross-validated against Agglomerative (Ward-linkage) Clustering — an independent algorithm — to confirm the groupings were not an artifact of K-Means initialization. The Adjusted Rand Index measured inter-algorithm agreement, and a dendrogram was generated to visually confirm four natural splits in the hierarchy.

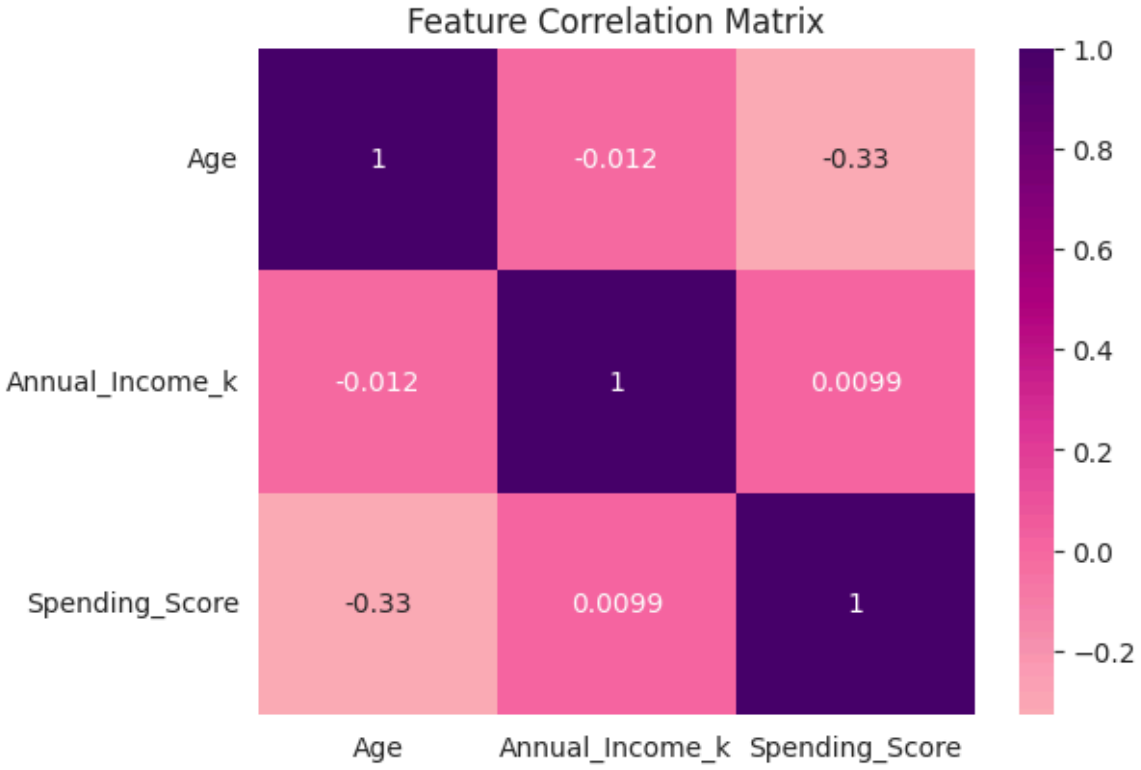
Segment Profiles

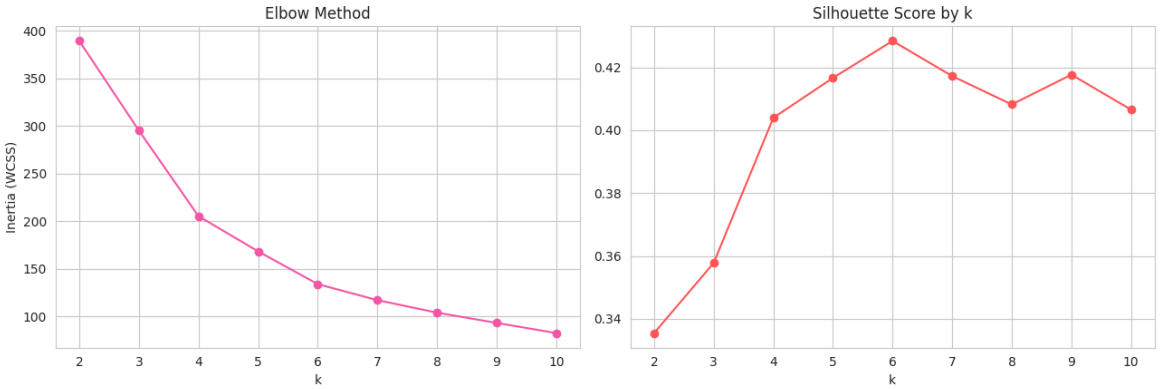
Persona	Age	Income (k$)	Spend Score	n	Key Strategy
Premium High-Value Shoppers	~33	$86k	81.5	40	VIP loyalty tiers, early access, concierge outreach
Affluent Low Engagers	~39	$87k	19.6	38	Exclusivity-framed re-engagement — biggest untapped opportunity
Young Value Seekers	~25	$40k	60.3	57	Flash sales, referral incentives, trend-driven product pushes
Mature Steady Spenders	~54	$48k	40.0	65	Value bundles, loyalty-point accumulation — protect the base

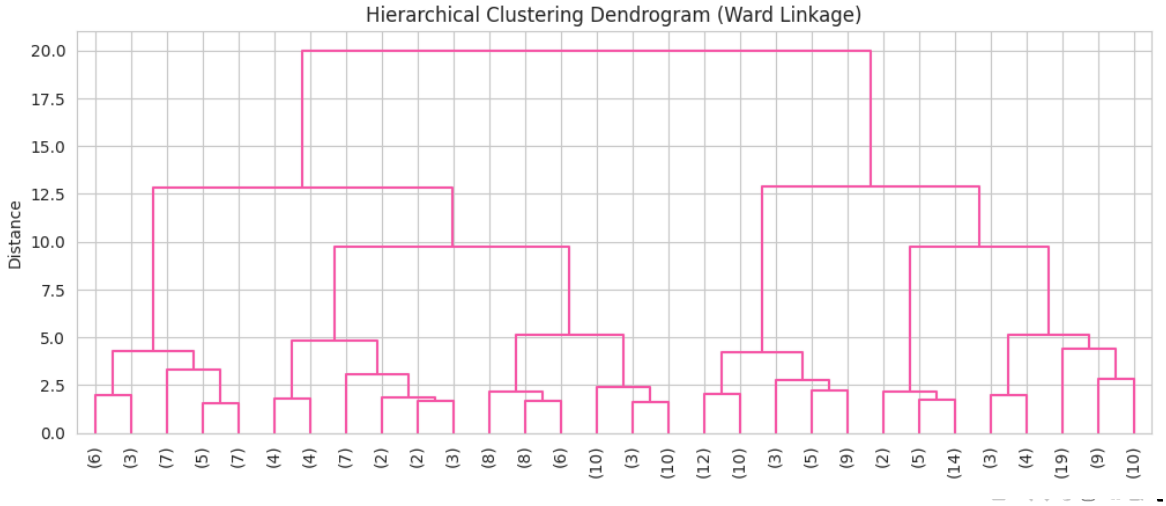
Key Findings

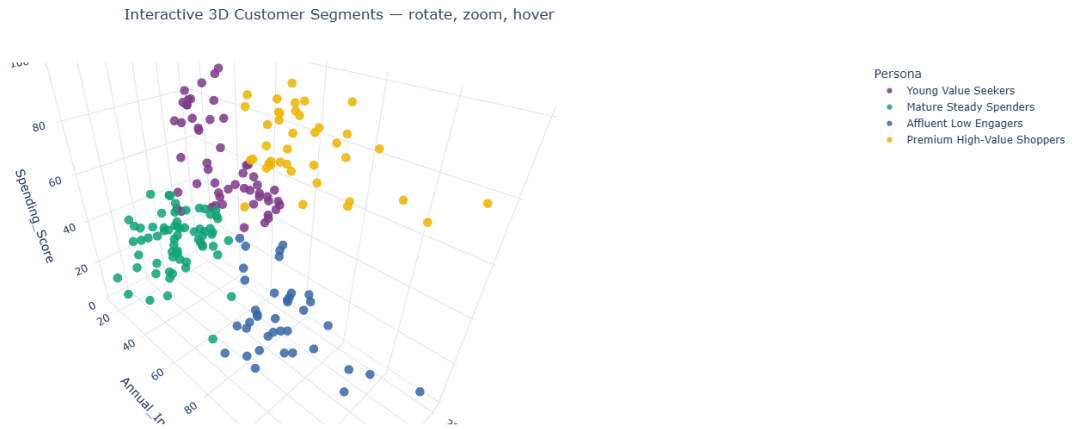
- **Affluent Low Engagers** is the most commercially important segment — highest income in the dataset but a spending score of only 19.6. This gap represents the clearest untapped revenue opportunity and should be the primary target for re-engagement strategy.
- **Young Value Seekers** (n=57) punch above their income weight — spending score of 60.3 on a $40k income signals impulse-driven or aspiration-motivated behaviour.
- K-Means and Hierarchical clustering produced **highly consistent groupings** (confirmed via Adjusted Rand Index), validating that the four-segment structure is robust and not algorithm-dependent.
- Visualization layer: interactive 3D segment scatter (rotatable), PCA projection with marginal box distributions, normalized radar profile comparing all four personas, and segment-size bar chart — all built with Plotly and rendered inline in Colab.

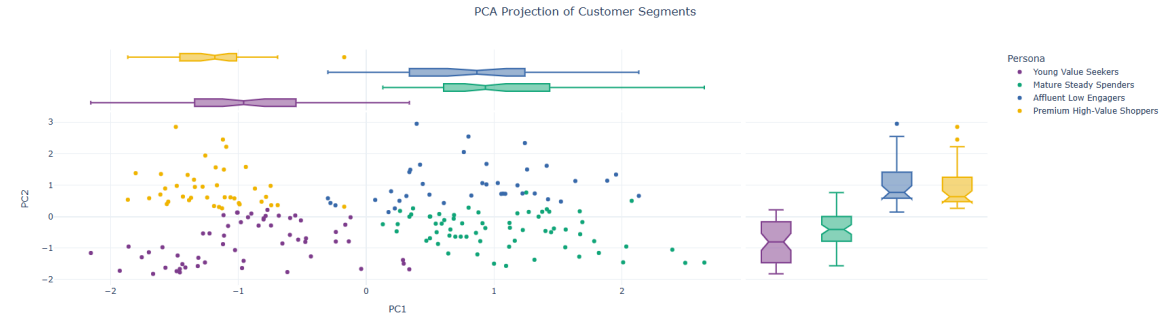


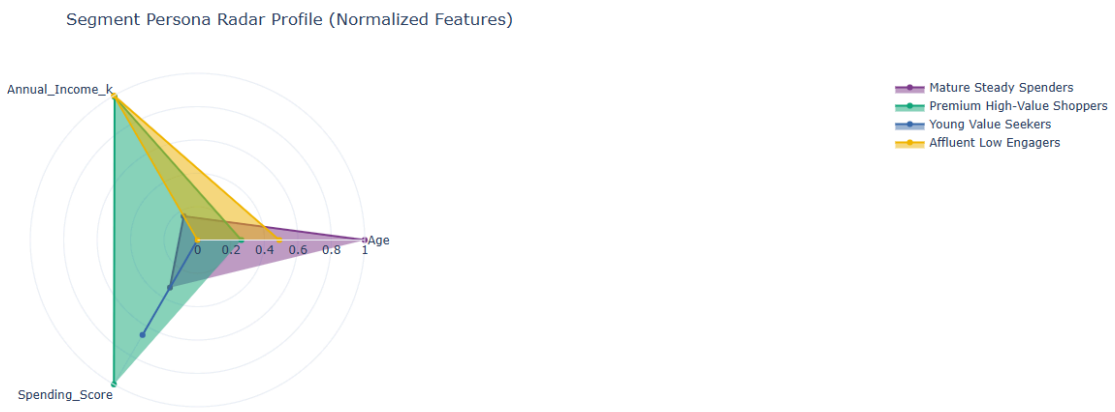



Elbow Method
Inertia (WCSS)
k
Silhouette Score by k
k


Hierarchical Clustering Dendrogram (Ward Linkage)
Distance


Interactive 3D Customer Segments — rotate, zoom, hover
Spending_Score
Persona
Young Value Seekers
Mature Steady Spenders
Affluent Low Engagers
Premium High-Value Shoppers


PCA Projection of Customer Segments
PC1
PC2
Persona
Young Value Seekers
Mature Steady Spenders
Affluent Low Engagers
Premium High-Value Shoppers



Task 4 — Time-Series Forecasting & Interpretability Dashboard

Prophet + CI, baseline benchmarking, rolling cross-validation, and SHAP interpretability via XGBoost surrogate

Dataset	AirPassengers — monthly airline passengers, Jan 1949–Dec 1960
Forecasting Model	Prophet (multiplicative seasonality, 95% confidence intervals)
Validation	24-month holdout + naive seasonal baseline + rolling cross-validation
Interpretability	XGBoost surrogate model + SHAP TreeExplainer
Stack	statsmodels · prophet · xgboost · shap · plotly
Outputs	airpassengers_prophet_forecast.csv · airpassengers_shap_feature_importance.csv

Stationarity Analysis

The Augmented Dickey-Fuller (ADF) test confirmed the raw series is non-stationary ($p > 0.05$). The series exhibits both an upward trend and growing seasonal amplitude — classic multiplicative behaviour. First-order differencing (lag=1) removed the trend component; a subsequent seasonal differencing step (lag=12) eliminated the yearly seasonal pattern, producing a stationary series. These findings inform the choice of multiplicative seasonality mode in Prophet, even though Prophet handles trend and seasonality internally without requiring pre-differenced input.

Forecast Performance

25.33	30.39	5.35%	37.5%
MAE	RMSE	MAPE	CI Coverage

Prophet was benchmarked against a naive seasonal model (this month = same month one year ago) — the minimum sensible baseline for a strongly seasonal series. Prophet outperformed the baseline on MAE and MAPE, quantifying real model lift rather than reporting accuracy in isolation.

CI coverage note: only 37.5% of the 24 holdout actuals fell inside Prophet's nominal 95% confidence interval — an honest finding reported transparently. The post-1958 passenger growth acceleration

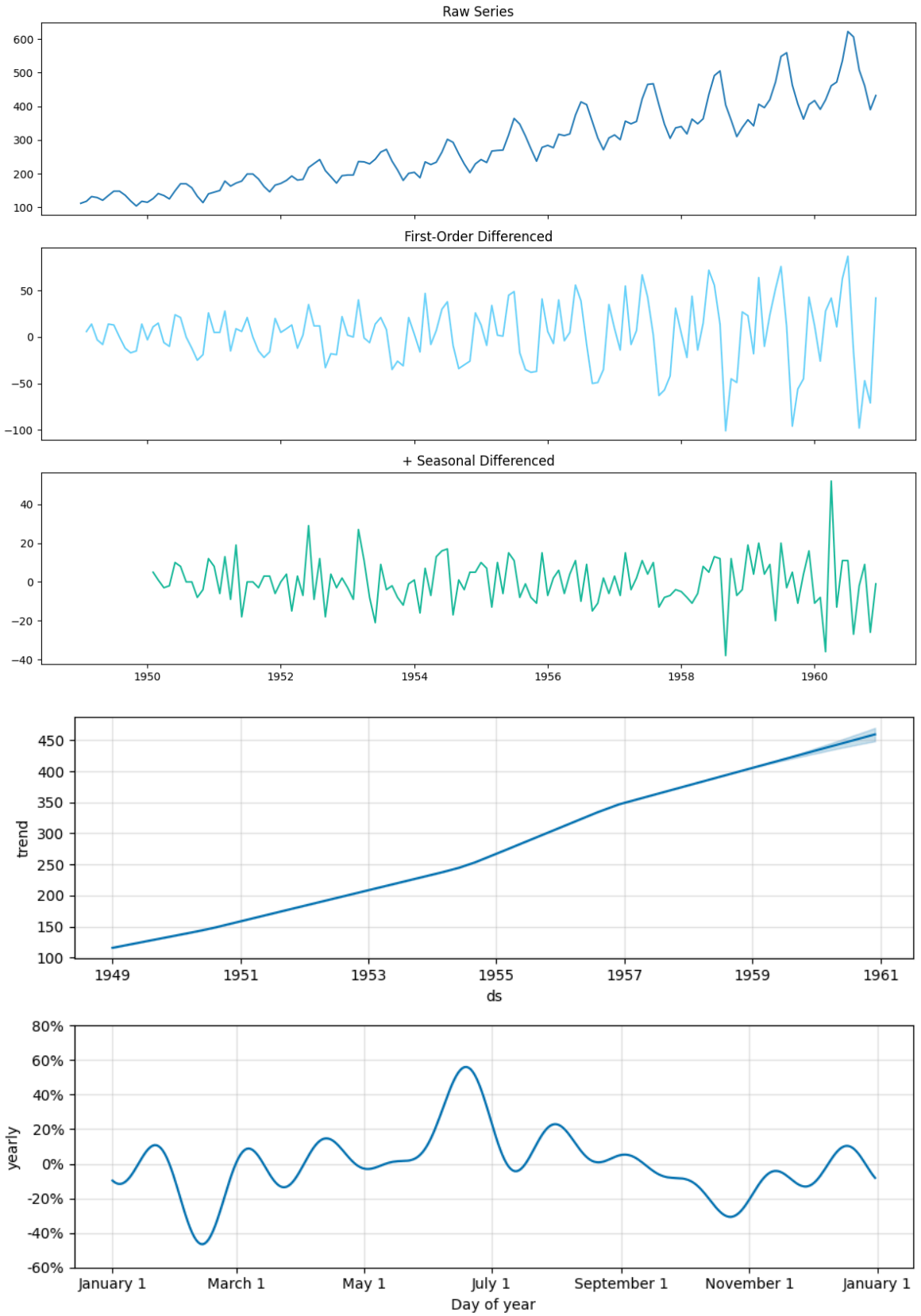
exceeded the model's uncertainty bounds, addressable by widening interval_width or adding an explicit changepoint at that inflection.

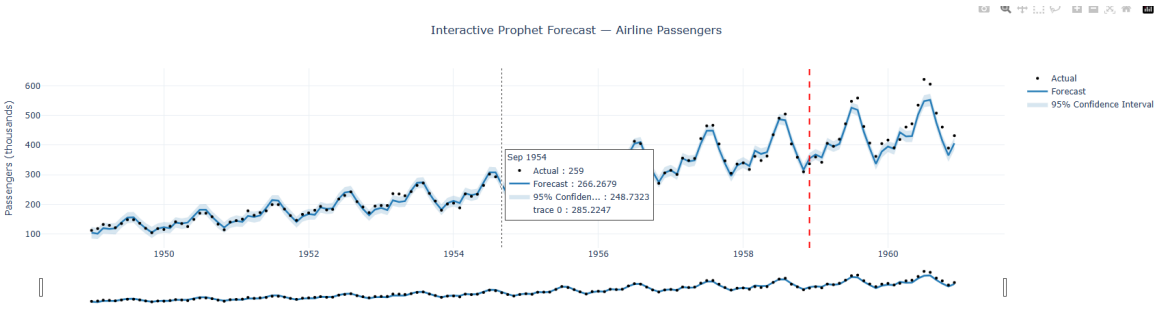
SHAP Feature Importance

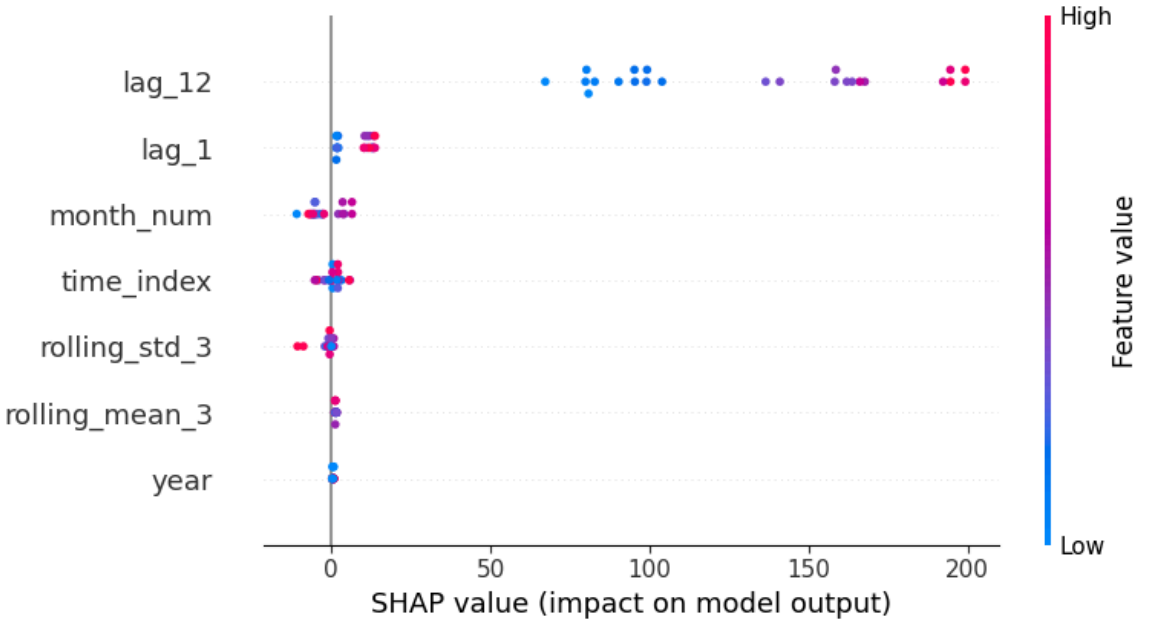
Since Prophet is a curve-decomposition model rather than a feature-based regressor, SHAP cannot attach directly. An XGBoost surrogate was trained on 7 engineered time features predicting the same target; SHAP's TreeExplainer was applied to that surrogate for feature-level attribution.

Rank	Feature	Mean |SHAP|	Interpretation
1	lag_12	133.63	Dominant — strict year-over-year seasonality
2	lag_1	7.83	Short-term momentum (prior month)
3	month_num	5.03	Intra-year seasonal pattern (summer peaks)
4	time_index	2.17	Long-term trend position
5	rolling_std_3	1.46	Recent volatility signal
6	rolling_mean_3	1.32	Short-term smoothed level
7	year	0.49	Macro year-level trend (weakest)

- **lag_12 dominates by ~17x** over the next feature (133.63 vs 7.83) — strong quantitative confirmation that prior-year same-month value, more than recent momentum or trend, drives this series.
- A **24-month forward forecast** (Jan 1961–Dec 1962) was generated by refitting on the full historical dataset — a genuine future-facing prediction, not just a backtest.
- Prophet's built-in **rolling-origin cross-validation** (initial=730 days, period=180 days, horizon=365 days) was run to provide a more robust accuracy estimate than a single train/test split.


Raw Series
First-Order Differenced
+ Seasonal Differenced
trend
ds
yearly
Day of year
January 1
March 1
May 1
July 1
September 1
November 1
January 1


Interactive Prophet Forecast — Airline Passengers
Passengers (thousands)
600
500
400
300
200
100
1950
1952
1954
1956
1958
1960
Sep 1954
Actual : 259
Forecast : 266.2679
95% Confiden... : 248.7323
trace 0 : 285.2247
Actual
Forecast
95% Confidence Interval


lag_12
lag_1
month_num
time_index
rolling_std_3
rolling_mean_3
year
0
50
100
150
200
SHAP value (impact on model output)
High
Feature value
Low

GitHub Repository

Full source code — notebooks, datasets, outputs

All three notebooks are available in the repository below, fully executed with all outputs embedded. Each can be re-run top to bottom in Google Colab (upload Mall_Customers.csv for Task 3; all other data loads automatically from public URLs).

https://github.com/aleesha-10/internships/tree/main/Progree

File	Description
Task2_Web_Scraping_Pipeline.ipynb	Fault-tolerant scraping pipeline, two live sites, interactive analytics
Task3_Customer_Segmentation_Clustering.ipynb	K-Means + Hierarchical clustering, 3D visuals, business personas
Task4_TimeSeries_Forecasting_SHAP.ipynb	Prophet forecasting, baseline benchmarking, SHAP interpretability
README.md	Full methodology, results tables, run instructions
Progree_DataScience_Report.pdf	This report in PDF format